

Contents

AI Arena — Platform Implementation Plan

Status at a glance (2026-04-26)

Phase 1 — Foundation: SDK + botInterface + XO Refactor

Phase 2 — Platform Consolidation: Rebrand and Navigate

[x] Flash alerts toggle saves

Phase 3 — Frontend Retirement, Tables Page, Platform Shell

Phase 3.7a — Pre-prod cleanups

Phase 3.8 — Multi-Skill Bots

Phase 4 — Connect4

Phase 5 — Poker

Phase 6 — Pong

Phase 7 — External Developer Onboarding

Cross-Cutting Items (any phase)

1

1

2

4

5

5

8

10

13

14

14

15

15

AI Arena — Platform Implementation Plan

Related: See Platform_Architecture.md for the decisions and rationale behind this plan.

Phases are sequential; items within a phase can run in parallel unless noted.

Status at a glance (2026-04-26)

Phase	Status	Notes
1 - Foundation (SDK + botInterface + XO refactor)	done (71/71)	Complete. (Four registry-switch items relocated to Phase 7.2.)
2 - Platform Consolidation (rebrand + nav)	done (44/44)	Complete.
3 - Frontend Retirement, Tables Page, Platform Shell	done (58/58)	3.0-3.5 complete. (3.6 multi-seat shell relocated to Phase 5.0; 3.7 rendered-table polish relocated to Cross-Cutting.)
3.7a - Pre-prod cleanups	done (25/27)	3.7a.1 (template split), .2 (bot-name uniqueness), .3 (profile route), .4 (OAuth note), .6 (auto-drop audit) all shipped. Open: .5 cosmetic built-in-bot polish (optional). (Frontend follow-up from .2 relocated to 3.8.3.)
3.8 - Multi-Skill Bots	done (25/25)	All three sprints landed: 3.8.A (backend reshape + Profile bot-card + AddSkillModal + inline name check + mixed-list disambiguation), 3.8.B (Gym sidebar drilldown, Profile→Gym nav, in-Gym Add-Skill, repointBotPrimarySkill), 3.8.C (identity-scoped picker payload, server-side (botId, gameId) resolution at match start with hard NO_SKILL 400, tournament reg validation, bracket/participant disambiguation, Playwright lifecycle e2e). Ready for production cut.
4 - Connect4	not started (0/20)	Blocked on 3.8.
5 - Poker	not started (0/24)	Blocked on 3.8 + 4. (Now includes 5.0 multi-seat shell, formerly 3.6.)

Phase	Status	Notes
6 - Pong	not started (0/18)	Blocked on 5's real-time spike review.
7 - External Developer Onboarding	not started (0/11)	Long-tail; no code dependencies. Now includes 7.2 — the four XO registry-switch items relocated from 1.4.

Phase 1 — Foundation: SDK + botInterface + XO Refactor

Goal: Establish the game contract and bot contract, publish XO as the reference implementation, validate both loading paths. Nothing else is built on the platform until this phase is complete.

1.1 Game SDK contract

- ☒ Define the GameContract TypeScript interface (meta + default component export)
- ☒ Define all meta fields including supportsBots, supportsTraining, supportsPuzzles, builtInBots
- ☒ Define the GameSDK interface: submitMove, onMove, signalEnd, getPlayers, getSettings, spectate, getPreviewState, getPlayerState
- ☒ Implement the platform-side SDK provider (creates the sdk object passed into every game)
- ☒ Write SDK developer documentation (contract spec, method signatures, example usage) — doc/Game_SDK_Developer_Guide.md

1.2 botInterface contract

- ☒ Define the BotInterface TypeScript interface
- ☒ Define all methods: makeMove, getTrainingConfig, train, serializeState, deserializeMove
- ☒ Define personas array structure
- ☒ Define GymComponent prop contract (what the platform passes in)
- ☒ Define puzzles array structure
- ☒ Implement the platform-side bot dispatcher (calls botInterface.makeMove() server-side for bot turns)
- ☒ Write botInterface developer documentation — covered in doc/Game_SDK_Developer_Guide.md

1.3 GitHub Packages registry

- ☒ Create @callidity npm scope on GitHub Packages — GitHub org callidity created
- ☒ Configure CI to authenticate with GitHub Packages for publish and install — ci.yml updated with registry-url + NODE_AUTH_TOKEN
- ☒ Document the publish workflow for internal use — .github/workflows/publish-packages.yml
- ☒ Add .npmrc configuration to all relevant packages — root .npmrc + packages/sdk/.npmrc; game packages get their own .npmrc when created

1.4 XO refactor into @callidity/game-xo

- ☒ Extract XO game logic into a standalone package under packages/game-xo/
- ☒ Implement full meta export including supportsBots: true, supportsTraining: true, supportsPuzzles: true, builtInBots
- ☒ Refactor XO game component to receive { session, sdk } props only — remove all direct platform calls
- ☒ Replace direct socket calls with sdk.submitMove / sdk.onMove
- ☒ Replace game-over logic with sdk.signalEnd
- ☒ Implement sdk.getPreviewState() — lightweight board snapshot
- ☒ Implement sdk.spectate() — live move feed for spectators
- ☒ Implement focused vs chrome-present rendering modes — derived from session.isSpectator
- ☒ Add escape affordance for focused mode (floating “Back to Arena” button)
- ☒ Implement botInterface.makeMove() — wraps existing minimax and ML bot logic
- ☒ Implement botInterface.personas — existing named bot personalities

- ☒ Implement `botInterface.getTrainingConfig(), train(), serializeState(), deserializeMove()`
- ☒ Migrate `GymComponent` into the package as `botInterface.GymComponent`
- ☒ Migrate puzzle content into the package as `botInterface.puzzles`
- ☒ Verify platform loads XO via `React.lazy(() => import('@calliduity/game-xo'))` — bundled path (builds clean, 37KB separate chunk)
- ☒ Document the registry switch mechanism — `doc/Registry_Switch_Guide.md`
- ☒ Run full regression — all XO functionality works through both loading paths (bundled path verified: PvP, win/draw/loss detection, marks, scores, reactions, forfeit, rematch; split-out URL path lives in Phase 7 — see “XO as the registry-switch reference”)
- ☒ Move `ruleBasedImplementation` from `backend/src/ai/ruleBased.js` into `packages/ai` so rule-based bot personas work in `botInterface.makeMove()`

1.5 Replay and live view abstraction

- ☒ Design game state reconstructor (applies move array to initial state in sequence) — `useReplaySDK.reconstructStates()`
- ☒ Update game renderer to accept either live socket feed or recorded move array — `useReplaySDK` provides same `{ session, sdk }` interface as `useGameSDK`
- ☒ Implement replay controls: play/pause, step forward/back, scrub, variable speed — `ReplayPage + ReplayControls`
- ☒ Implement live view mode: input disabled, observer status signalled in UI — derived from `session.isSpectator: true`; `ReplayPage` delivers via fake spectate SDK
- ☒ For Pong (future): confirm sampled snapshot approach (100ms intervals) works within this abstraction — confirmed in Phase 1.8 spike (`doc/Pong_Spike_Findings.md` §“Replay storage”)
- ☒ Test: replay a completed XO game end-to-end — play a game, visit `/replay/:id`

1.6 Replay retention infrastructure

- ☒ Add `moveStream` storage to game records (separate from result) — `Json?` field on `Game`; populated by `roomManager + socketHandler + botGameRunner`
- ☒ Add `isTournament` Boolean flag to game records — backfilled from `tournamentId IS NOT NULL`
- ☒ Implement admin-configurable TTL settings (casual TTL, tournament TTL, default 90 days) — `SystemConfig` keys `replay.casualRetentionDays / replay.tournamentRetentionDays`
- ☒ Build scheduled purge job — deletes expired move streams, retains game results permanently — `replayPurgeService.js`, 24h interval
- ☒ Migrate existing `replayRetentionDays` from `Tournament` model to new admin TTL config — defaults set to 90 days in `SystemConfig`
- ☒ Remove `replayRetentionDays` field from `Tournament` model — migration `20260413120000_replay_stream`
- ☒ Add retention settings to admin panel — `ReplayConfigPanel` in `AdminDashboard`

1.7 Schema migration — Skills, per-game ELO, terminology

MLModel → **BotSkill** - [x] Rename `MLModel` Prisma model to `BotSkill` - [x] Add `botId` `String` field (userId of the owning bot) - [x] Add `gameId` `String` field - [x] Add unique constraint on (`botId`, `gameId`) - [x] Rename `qtable` `Json` field to `weights` `Json` - [x] Add `algorithm` `String` field — records which algorithm produced the weights so `makeMove` can deserialize correctly (e.g. 'qlearning', 'alphazero') - [x] Migrate existing `MLModel` records — set `gameId` = 'xo', set `botId` from `createdBy`, set `algorithm` = 'qlearning' - [x] Remove `eloRating` from `BotSkill` (ELO moves to `GameElo`) - [x] Run and verify migration against staging DB — `20260414000000_phase17_bot_skill_game_elo + 20260414120000_fix_game_elo_bot_skill_column_maps` shipped with the v1.3.0-alpha-1.x staging cycle and are live in current staging

Per-game ELO - [x] Create `GameElo` model: `{ userId, gameId, rating Float, gamesPlayed Int }` - [x] Migrate existing `eloRating` values from `User` to `GameElo` entries for XO (humans and bots) - [x] Migrate existing `eloRating` values from old `MLModel` to `GameElo` entries - [x] Remove `eloRating` field from `User` model - [x] Update all ELO read/write logic to use `GameElo` - [x] Update leaderboard and profile pages to display per-game ratings

Terminology rename throughout codebase - [x] Rename `mlService.js` → `skillService.js`, update all imports - [x] Replace “brain” / “model” with “skill” in all UI copy, API responses, and comments - [x] Rename API routes: `/api/ml/*` → `/api/skills/*` (both mounted; `/api/ml` kept for backward compat) - [x] Update admin panel labels - [x] Update Gym UI — algorithm display, checkpoint display, ELO references removed from `BotSkill`

1.8 Pong real-time spike

Run in parallel with 1.4. Real-time architecture findings must be in hand before Connect4 is complete.

- ☒ Build minimal Pong prototype using tight WebSocket loop (Socket.io)
 - ☒ Measure game feel, latency, and server load at simulated concurrent tables
 - ☒ Decision point: confirm WebSocket loop is sufficient or escalate to WebRTC evaluation — **WebSocket confirmed sufficient**
 - ☒ Document findings and recommended approach — doc/Pong_Spike_Findings.md
-

Phase 2 — Platform Consolidation: Rebrand and Navigate

Goal: The unified AI Arena identity, nav, and journey live on landing/. Frontend retirement is split out into Phase 3.0 since it's the natural prerequisite for the Tables-page work.

2.1 Unified visual identity

- ☒ Remove XO-specific theming (mountain background, teal/blue per-site identity) — Colosseum background live; no per-site theming
- ☒ Align to AI Arena design language (Colosseum + slate blue) — live on staging
- ☒ Audit all hardcoded “XO Arena” strings — replace with “XO” or “AI Arena” — AppNav “XO Arena” → “XO”, BotProfilePage “XO Arena (built-in)” → “AI Arena (built-in)”, page title “AI Arena”
- ☒ Verify shared packages/nav renders consistently — verified on staging, desktop + mobile

2.2 Primary navigation

- ☒ Update nav to: Tables · Tournaments · Rankings · Profile · About — navItems.js updated, verified
- ☒ Remove Games dropdown — confirmed gone
- ☒ Fold FAQ into About page (/about#faq or tabbed section) — About page has “Help” section linking to /faq; FAQ no longer in primary nav, accessed through About
- ☒ Update packages/nav/src/navItems.js with new structure — done
- ☒ Verify desktop and mobile (hamburger) nav — verified on staging

2.3 Onboarding journey update

- ☒ Audit all journey steps for references to old nav items, site names, or structural flows — all 8 step titles and hrefs are correct; all routes are internal
- ☒ Update step copy and instructions to reflect new nav and AI Arena identity — JourneyCard.jsx STEPS titles confirmed correct; JOURNEY_DEFAULT_SLOTS step 7 slot corrected from play_my_bot → tournaments to match “Enter a tournament” step
- ☒ Update any journey step that references “XO Arena” to “XO” or “AI Arena” — JourneyCard.jsx badge label updated
- ☒ **Re-wire journey steps whose completion is triggered by a route visit** — slotActions.js XO-section slots converted to internal routes; JourneyCard.jsx steps 3, 4, 6 converted from external cross-site links to internal <Link>
- ☒ Update site badges on journey cards if needed — all steps use site: 'platform' → “AI Arena” badge; correct
- ☒ QA full journey flow end-to-end — verify every step can be completed

2.x QA Checklist

Signed off 2026-04-15 against staging v1.3.0-alpha-1.06.

Core navigation

- ☒ Landing home page loads with correct branding (“AI Arena” / “XO”, no “XO Arena”)
- ☒ All 5 nav items visible and route correctly: Tables · Tournaments · Rankings · Profile · About
- ☒ No broken links or 404s in the main nav
- ☒ Mobile hamburger nav opens and all items are reachable

Cross-site links removed

- ☒ Home page “Play” button routes to /play internally (no redirect to external site)
- ☒ Profile page stats and bot links route internally
- ☒ Journey card steps 3, 4, 6 route internally — no cross-site navigation
- ☒ Guide slot actions route internally

Ported pages (new on landing)

- ☒ /gym loads the Gym page
- ☒ /gym/guide loads the Gym Guide page
- ☒ /puzzles loads the Puzzle page
- ☒ /rankings loads the Rankings page
- ☒ /stats loads the Stats page
- ☒ /bots/:id loads the Bot Profile page

Phase 2.1 — Branding

- ☒ No “XO Arena” text visible anywhere in the UI
- ☒ Home page, About page, and welcome modal use “AI Arena” / “XO” correctly

Phase 2.2 — Navigation

- ☒ Nav shows: Tables · Tournaments · Rankings · Profile · About
- ☒ No Games dropdown visible
- ☒ FAQ content accessible via About page
- ☒ Rankings nav item routes to /rankings on landing (not cross-site)

Phase 2.3 — Journey

- ☒ Journey opens correctly for new users
- ☒ All journey steps can be completed end-to-end
- ☒ No journey step links to the old frontend domain

Auth flows

- ☒ Sign-in modal opens
- ☒ Google OAuth sign-in works end to end
- ☒ Signed-in state persists on refresh

Settings

- ☒ Settings page loads when signed in
- ☒ Notification preference toggle saves

•

[x] Flash alerts toggle saves

Phase 3 — Frontend Retirement, Tables Page, Platform Shell

Goal: Retire the legacy XO frontend service, then ship the Tables page (the new front door) and the platform shell that loads any registered game.

3.0 Retire the XO frontend service

Prerequisite: Phase 2 complete — landing fully owns the unified AI Arena UI; XO loads correctly through the platform shell on landing.

- ☒ Confirm @callidity/game-xo loads and plays correctly via the platform shell — verified end-to-end on staging at v1.3.0-alpha-1.06

- ☒ Confirm Gym and Puzzles render correctly via `botInterface.GymComponent` and `botInterface.puzzles` — Gym, Gym Guide, and Puzzles all render on landing
- ☒ Remove frontend/ service from the monorepo — deleted in a4ad867, 193 files / 41 MB
- ☒ Remove frontend service from Fly.io (xo-frontend-staging, xo-frontend-prod) — both destroyed via `flyctl apps destroy` on 2026-04-15
- ☒ Remove frontend from `docker-compose.yml` — 73067c8
- ☒ Remove frontend deploy steps from `.github/workflows/deploy-staging.yml` and `deploy-prod.yml` — dc61aa7
- ☒ Update e2e smoke tests — drop `BASE_URL=https://xo-frontend-staging.fly.dev` from the harness — a79abe5; playwright `baseUrl` now defaults to `LANDING_URL`
- ☒ Update all remaining internal references from XO frontend URL to AI Arena URL — `landing/server.js` /xo proxy removed (10e1dc2); `landing/Dockerfile VITE_XO_URL` arg removed; `generate-training-guide-pdf.yml` re-pointed to `landing/public/`
- ☒ Final QA: staging and prod both verified clean at v1.3.0-alpha-1.07+ with 12/12 smoke tests passing

3.1 Table data model (backend) — complete

Signed off 2026-04-15 against staging v1.3.0-alpha-1.08.

- ☒ Create Table Prisma model with all required fields — 85bd20d; `TableStatus` enum (FORMING/ACTIVE/COMPLETED), `seats` Json, `previewState` Json, `isPrivate`, `isTournament`; indexes on `status`, `gameId`, `createdById`, `isTournament`
- ☒ Run and verify migration — 20260416000403_phase3_tables applied to local dev DB; Prisma Client regenerated
- ☒ Add table CRUD endpoints (create, list, get, join, leave) — bd5eacc; 5 routes on `/api/v1/tables`; 26 vitest cases (create validation, list filters, auth gate, get-one-private, join idempotency, leave idempotency)
- ☒ Private table share link — table accessible at `/tables/[id]`; private tables excluded from default list but reachable by direct URL (GET `/api/v1/tables/:id` always works)
- ☒ Add notification bus events: `table.created`, `player.joined`, `spectator.joined`, `table.empty` — ddf3969; 4 event types in `REGISTRY + PREF_DEFAULTS`; wired into `create/join/leave` routes + `spectator.joined` from `presence`
- ☒ Add presence tracking per table (who is watching) — 8b75e17; `tablePresence.js` module (`addWatcher/removeWatcher/getPresence`) + `table:watch/table:unwatch` socket events + `disconnect` cleanup; 12 unit tests
- ☒ Update tournament service to set `isTournament: true` on generated tables — 8be204e (Option A: Tables alongside `TournamentMatch`); `tournamentBridge` creates Table row at `match:ready`, marks `COMPLETED` at `match:result`; 8 tests. Phase 3.4 commits to collapsing the dual-write.

3.2 Tables page (frontend)

- ☒ Build Tables page at `/tables` — 5a6f5a2; `TablesPage.jsx` with `status` + `game` filters, empty state
- ☒ Live list of open public tables — private tables hidden from list, accessible by direct URL only — 5a6f5a2; `api.tables.list()` excludes private by default, GET `/tables/:id` always works
- ☒ Table card shows: game type, table icon, status, players seated, spectator count, `previewState` thumbnail — 5a6f5a2 + 90289c5; card shows game label, seat strip (filled/empty dots), status badge, seated count; spectator count lands on `TableDetailPage` via `table:presence`; `previewState` thumbnail deferred to 3.5 (needs game-specific renderer + the rendered table surface from Phase 3.5)
- ☒ forming state: table card shows empty seats waiting to fill — 5a6f5a2; seat strip dots show filled vs outline
- ☒ Create table flow: choose game, set private/public, configure settings — 5a6f5a2; `CreateTableModal` with game picker + private/public toggle
- ☒ Join table flow: click table — sit down (if seat available) or spectate — 5a6f5a2; `TableDetailPage` shows seats + Take a seat / Leave seat buttons
- ☒ Empty state: helpful prompt when no tables are open — 5a6f5a2; signed-in users see “Create table” CTA, guests get “sign in” prompt
- ☒ Real-time updates — table list reflects new tables, seat changes, status changes without page refresh — 90289c5; both pages subscribe to `guide:notification` + debounced `refetch`; table events routed out of the user notification stack
- ☒ Bot-vs-bot tables always appear in public list — bot tables are just regular tables with bot user IDs seated; not filtered out by the list
- ☒ **Instrument critical resources in admin health.** — 084ac0f; 5 new tiles on `AdminHealthPage` (Tables Forming/Active/Completed/Stale + Table Watchers), backed by `takeTablesSnapshot()` in `resourceCounters.js`. Stale-FORMING surfaced as a metric (not yet a leak alert — legitimate for private tables waiting to be shared). Other critical resources were already covered (notif queue depth, scheduler pending/running/failed, dispatcher heartbeat, pending PvP match map).

- ☒ **Post-3.2 polish (pre-3.5).** Landed after the initial Tables page shipped, while 3.3 was in flight:
 - ListTable refactor so the list scales to 100+ tables with sticky header + fitViewport scroller
 - Mobile: truncate long game names instead of wrapping so rows stay one line
 - Symmetric seat click on detail page: empty seat → take it; own occupied seat → leave it
 - Server placement honors seatIndex on join (previously always picked first empty seat)
 - Creator-only delete for non-tournament, non-ACTIVE tables (DELETE /api/v1/tables/:id) + table.deleted bus event + client redirect
 - Creator can see their own private tables in /tables; other users still cannot
 - ShareTableButton (icon + full variants) on list row and detail page — copies /tables/:id URL with clipboard + execCommand fallback
 - Loading skeleton replaced by a single centered spinner (quieter on filter changes)

3.3 Platform shell and game loading — complete

Signed off 2026-04-16 against local dev; will verify on staging at the next /stage.

- ☒ Build platform game shell — wraps any loaded game, manages focused vs chrome-present mode — 459e665; landing/src/components/platform/PlatformShell.jsx
- ☒ Shell automatically detects active player status and sets rendering mode — 459e665; selectDefaultMode({isSpectator, phase}): seated+playing → focused, otherwise → chrome-present
- ☒ Focused mode: full viewport, chrome hidden, escape affordance visible — 459e665; semi-transparent ← Back and expand buttons top-corners
- ☒ Chrome-present mode: nav + table context sidebar visible, game in content area — 459e665; grid layout with game column + 260px sidebar
- ☒ Table context sidebar: table info, seated players, spectator count, presence indicators — 459e665; game title, status badge, watching count, seated players list with BOT badge
- ☒ Game-specific tabs: Gym tab (if supportsTraining: true), Puzzles tab (if supportsPuzzles: true) — 459e665; driven off meta flags, games opt in declaratively
- ☒ Gym tab renders botInterface.GymComponent — implemented as deep-link to /gym?gameId=xo rather than embedding the GymComponent inside the shell. Rationale: the full Gym page at /gym already wraps botInterface.GymComponent with the platform training infrastructure (socket progress events, session store, etc.); re-embedding it inside the shell would duplicate that wiring. A future refinement could swap the shell's main area for an inline embed if the navigation feels disruptive — trivial change given the tab-link plumbing is already in place.
- ☒ Puzzles tab renders botInterface.puzzles content — same link-based approach, deep-linked to /puzzles?gameId=xo.
- ☒ Load game via React.lazy(() => import('@callidity/game-xo')) through shell — 2ab5600; PlayPage's lazy import passes through the shell's Suspense boundary
- ☒ Verify XO plays correctly through the new shell including Gym and Puzzles tabs — 2ab5600 routes /play through the shell; 13 component tests + 40 page tests cover the integration. TableDetailPage (e110f28) renders ACTIVE tables through the shell too, ready for Phase 3.4 to wire the game session in.

3.4 Retire in-memory Room layer (Tables become the only primitive) — complete

Signed off 2026-04-18 against v1.3.0-alpha-1.13 (dev). QA checklist: doc/QA_Phase_3.4.md.

Goal: Tables are THE source of truth for live game sessions. The in-memory roomManager and the dual-write between Table and TournamentMatch go away; the realtime layer reads/writes Tables directly.

Why this exists: Phase 3.1 chose **Option A** (additive — tournament bridge creates Table rows alongside TournamentMatch so the Tables page lights up immediately). That left two intentional sources of truth for game session state: the Table row and the in-memory Room (created by roomManager and used by socketHandler's room:* events). Phase 3.4 collapses them.

- ☒ Make Table canonical for game session state — Room fields moved into Table.previewState; lastActivityAt added as indexed column
- ☒ Table GC (idle cleanup) — tableGcService purges stale FORMING/COMPLETED/ACTIVE-idle tables on a periodic sweep
- ☒ Admin table management page — ListTable-based view in admin panel; force-delete with room:abandoned broadcast and reason: 'admin'
- ☒ Rewrite socketHandler.js room:* events to operate on db.table instead of roomManager

- ☒ Delete dual-write code in tournamentBridge.js; TournamentMatch records bracket position only
- ☒ Delete roomManager — removed from codebase
- ☒ Migrate idle/abandonment timers to Table-aware service
- ☒ Verify presence tracking works correctly with Tables as source of truth
- ☒ Tournament-match flow QA — no regressions

3.4a Tournament QA + seed bots (parallel to 3.4) — complete

Signed off 2026-04-18. Runs parallel to the Room→Table collapse; validates the tournament path and adds seed bot support.

- ☒ Full tournament QA checklist (doc/QA_Phase_3.4.md Section 8) — lifecycle, BOT_VS_BOT automated path, HVH match play, ELO isolation, odd-player bracket, auto-cancellation
- ☒ Tournament seed bots — admin-configured bot accounts auto-enrolled in every recurring tournament occurrence
 - TournamentSeedBot schema + migration (tournament_seed_bots table)
 - Tournament API: GET/POST/DELETE /api/tournaments/:id/seed-bots
 - Scheduler copies seed bots from template into each new occurrence (checkRecurringOccurrences)
 - botGameRunner: seed: botModelId prefix handled alongside testbot:/builtin:
 - 3 vitest cases covering enrollment, error recovery, empty-template case
 - QA checklist updated (Section 9 — 6 sub-sections)

3.5 Rendered table paradigm — minimum viable — complete

Goal: Ship the Medium rendered table for 2p sit-down and 2p head-to-head. XO validates sit-down; shell is ready for head-to-head when Pong arrives. See doc/Table_Paradigm.md for the full design decisions.

- ☒ Add meta.tableArchetype('sit-down' | 'head-to-head')+meta.orientations(['horizontal', 'vertical']) fields to GameContract. Defaults preserve existing behavior. (*tableArchetype: 'sit-down' shipped on the SDK contract and XO meta. The 'head-to-head' union and orientations field are deferred to Phase 4/6 — only relevant when Connect4 / Pong land.*)
- ☒ Evolve PlatformShell to render a <TableSurface> with positioned <Seat> slots. Board renders in the <TableCenter> rect.
- ☒ Seats are avatar + name, spatially positioned (no more sidebar-only seated list for the primary rendering).
- ☒ Forming → Playing transition: fade-in on the center when ACTIVE lands (Table_Paradigm §4.4 option B); respect prefers-reduced-motion.
- ☒ Relative POV (§4.2): caller at bottom / near-end; opponents arranged relative to caller.
- ☒ Spectator badge (§4.3): edge cluster with click-to-expand popover listing watcher names; sidebar list unchanged.
- ☒ End-of-game seat indication (§4.5): winner glow, loser muted, plus small outcome banner.
- ☒ Tournament context card in sidebar (§4.7) when isTournament = true.
- ☒ **Active table preview thumbnail.** Render a mini previewState snapshot (e.g., a small XO board with current marks) on the Tables list page for ACTIVE tables. Deferred from Phase 3.2 — requires the game-specific renderer that the <TableSurface> component provides.
- ☒ QA: XO still plays correctly via the new shell on both desktop and mobile. (*QA_Phase_3.4 closed 2026-04-22; archived at doc/archive/QA_Phase_3.4.md.*)

Phase 3.7a — Pre-prod cleanups

Goal: A narrow window of housekeeping items that are trivial while xo-**-prod apps sit pending with an empty xo-db-prod, and progressively more expensive once the system has real users. Ships before Phase 3.8 so the foundation under Multi-Skill Bots and Connect4 is clean.

Why now: Every item here scales in cost with data volume, or involves user-visible URL/auth behavior that breaks links after launch. Post-launch each one is 5–10× harder.

3.7a.1 Recurring tournaments — template vs occurrence split — done

Full design in doc/Tournament_Template_Refactor_Scope.md. Shipped in three stages on 2026-04-23 (commits 91a22a7 readers, f3aa667 endpoint split, a4b40f8 column drop).

- ☒ Add TournamentTemplate Prisma model (config fields + recurrence + paused); every Tournament becomes a pure occurrence with nullable templateId FK.
- ☒ Prisma migration drops Tournament.isRecurring, recurrenceInterval, recurrenceEndDate, recurrencePaused, autoOptOutAfterMissed. Adds templateId String? + FK to TournamentTemplate.id. (recurrenceStart / recurrenceRule were never on Tournament — the old schema used startTime as the anchor.)
- ☒ Point RecurringTournamentRegistration.templateId FK at TournamentTemplate.id (same field name, new target).
- ☒ New TournamentTemplateSeedBot table keyed (templateId, userId). Per-occurrence TournamentSeedBot rows remain for one-shot overrides; scheduler enrolls template seeds into every spawned occurrence.
- ☒ tournament/src/lib/recurringScheduler.js: reads templates via db.tournamentTemplate.findMany(...). Sets templateId on spawned occurrences; no longer writes isRecurring.
- ☒ tournament/src/lib/tournamentSweep.js: unchanged cancel / auto-drop logic still applies per-occurrence; no new semantics.
- ☒ tournament/src/routes/tournaments.js: POST /api/tournaments/admin/templates for recurrence config; POST /api/tournaments for one-shots (rejects isRecurring:true with 400).
- ☒ Admin UI — two tabs: **Tournaments** (occurrences, today's flat list) and **Templates** (recurring series, paused state, subscriber count). Public TournamentsPage.jsx unchanged — only lists occurrences.
- ☒ Tests — tournamentSweep + recurringScheduler + seedBotService vitest green; e2e tournament-template-create (happy path + validation + AUTO anchor fallback), tournament-template-clone (clone + reuse + built-in delete guard), tournament-seed-bots (§9a-e) all pass against local.
- ☒ doc/Platform_Architecture.md — tournament data-model section checked; stays at platform-architecture level, no stale Prisma specifics to regenerate.

3.7a.2 Bot displayName uniqueness — (c) hybrid — done

- ☒ **Decision (2026-04-23):** neither pure global uniqueness nor free-text — hybrid. Two partial unique indexes on users, both case-insensitive:
 - UNIQUE (LOWER("displayName")) WHERE "isBot"=true AND "botOwnerId" IS NULL — protects built-in / reserved names (Rusty/Copper/Sterling/Magnus). No user can create an orphan bot that collides with a built-in.
 - UNIQUE ("botOwnerId", LOWER("displayName")) WHERE "isBot"=true AND "botOwnerId" IS NOT NULL — one owner can't have two bots with the same name. Cross-owner collisions stay allowed (your Rusty vs my Rusty).
- ☒ Migration 20260423010000_bot_displayname_hybrid_unique applied to local DB. Cleanup step deleted 344 orphan test bots (E2E / stress-test leftovers); built-in roster untouched.
- ☒ POST /api/v1/bots returns 409 BOT_NAME_TAKEN on collision (either partial unique). Other P2002 collisions (email, etc.) fall through to the generic handler.
- ☒ 3 new vitest cases cover per-owner collision, built-in collision, and the non-name P2002 fall-through.

3.7a.3 Profile URL structure — done

- ☒ **Finding:** no shareable profile URL exists today — /profile renders the logged-in user's own profile. No collision risk from launch.
- ☒ **Decision:** when a public profile page is introduced (Phase 7 or earlier if needed), use /users/:username as the canonical URL. Record the decision here so it isn't re-litigated.
- ☒ Route reserved at landing/src/App.jsx:58 → landing/src/pages/PublicProfilePage.jsx stub. Future real PublicProfilePage replaces the stub without URL migration; existing /users/... links stay valid.

3.7a.4 OAuth prod redirect URLs — noted

Deferred to prod-bringup day. No code change in this phase.

- ☒ Cross-reference doc/Prod_Bringup_Runbook.md §4: when prod deploys for the first time, either (a) add https://aiarena.callidytv to the existing staging Google + Apple OAuth apps, or (b) register prod-specific apps. Runbook already captures both paths.
- ☒ No code change in this phase — pure provider-side config. Entry here exists so it's not forgotten in the Phase 4+ rush.

3.7a.5 Seeded built-in bot polish

Concrete sub-items (unpacked 2026-04-29 from the placeholder that previously lived in Future_Ideas.md):

- **Avatars** — `User.avatarUrl` exists but `backend/prisma/seed.js` does not set it for any built-in bot, so all four fall back to the generic UI default. Pick four images (commit under `landing/public/bots/<name>.png` or similar), set `avatarUrl` in the seed's create branch, and add it to the upsert's update branch so re-seeds repaint stale rows.
- **Bios** — there is **no bio/tagline field on User**. Adding one requires either (a) a small schema migration to add `User.tagline String?` (cleanest, also reusable for human accounts later), or (b) repurposing `BotSkill.description` — but that is per-skill, not per-bot, so it doesn't fit a "Rusty's character" line shown on a profile. Recommend (a).
- **Starting ELO offsets** — `GameElo` rows are created lazily on first match; nothing in the seed pre-creates them. Today the Rusty/Copper/Sterling/Magnus ladder is conveyed *only* by minimax tier (search depth), not by displayed ELO. To get a visible ladder before any user has played, the seed should upsert a `GameElo` row per bot for `gameId='tic-tac-toe'` (post-A1.5b slug cutover) with offsets like 1000/1200/1400/1600. Idempotent: only create if missing so re-seeds don't stomp live ELO.

Verdict: still low-value / cosmetic. No user has complained that the four bots look samey. Defer until a product pass surfaces something specific (or a marketing screenshot needs them differentiated).

- ☐ Add per-bot `avatarUrl` to `BUILT_IN_BOTS` in `backend/prisma/seed.js`. Source images live under `landing/public/bots/`.
- ☐ Schema: add `User.tagline String?` migration; populate per-bot taglines in seed.
- ☐ Seed `GameElo` rows for (`botId`, `gameId='tic-tac-toe'`) with a Rusty < Copper < Sterling < Magnus offset ladder. Create-if-missing only.
- ☐ Manual pass: open each bot's public profile + the rankings page; confirm the four are visibly differentiated.

3.7a.6 Admin metric — tournaments auto-dropped per period — done

Context: Phase 3.5 added silent deletion of unfilled bot-only tournaments (`tournamentSweep.autoCancel` → hard DELETE when all participants are bots). That hides clutter from the public tournaments list, but admins should still have visibility into "how often is this happening" — it's a health signal for scheduling / seed-bot tuning. The DELETE removes the tournament row itself, so a post-hoc COUNT is impossible; we need an audit row written *before* the delete.

- ☒ **Schema:** `TournamentAutoDrop` — { `id`, `originalTournamentId?`, `templateId?`, `name`, `game`, `minParticipants`, `participantCount`, `droppedAt` }. Append-only. Index on `droppedAt`. Migration `20260423160000_tournament_auto_drops`.
- ☒ **Sweep hook:** `tournamentSweep.js:autoCancel()` bot-only branch writes the audit row before `db.tournament.delete(...)`. Non-fatal on insert failure — delete still proceeds.
- ☒ **API:** GET `/api/v1/admin/tournaments/auto-dropped?period=day|week|month` in `backend/src/routes/admin.js`. Returns { `period`, `since`, `count`, `items` } (items capped at 20, newest first). Unknown periods → 400.
- ☒ **Admin UI:** `AutoDropWidget` on `/admin/tournaments` — headline count + Day/Week/Month toggle + last 10 items with `participantCount/minParticipants · timestamp`.
- ☒ **Tests:** vitest — `autoCancel` bot-only writes audit (sibling + zero-participant cases); human-present cancel does NOT; endpoint covers default period, day override, and 400 on unknown period.
- ☒ **Retention:** 90-day prune inside the tournament sweep tick, gated by a once-per-day timer so it only runs the `deleteMany` once per 24h.

Phase 3.8 — Multi-Skill Bots

Goal: A bot is an identity (alter ego). Skills are what the bot knows how to play. One `User` row carrying `isBot: true` can hold multiple `BotSkill` rows (one per game). Users pick the bot; the game context chooses the skill.

Why this ships before Phase 4: Connect4 is the forcing function — without this in place, "Rusty" would become a separate second bot for Connect4 instead of a second skill on the same bot. The schema already anticipates this (`BotSkill` is keyed (`botId`, `gameId`) unique from Phase 1.7); only the flows and UI need to catch up.

Decisions locked 2026-04-23: - Name: *Multi-Skill Bots*. - Bot create is two-step: create the bot (name + avatar + competitive flag), then *Add a skill* separately. No implicit first-skill bundling. - Pickers are identity-scoped: user picks *Rusty* in Play or tournament registration; the game context determines which of Rusty's skills runs. No "Rusty

(XO)” / “Rusty (C4)” duplicate entries. - `User.botModelId` stays as a nullable “primary/last-trained skill” pointer for UI convenience. Source of truth for skills is `BotSkill.findMany({ where: { botId } })`.

Sprint plan locked 2026-05-02 — production-readiness shape. Goal: ship a complete platform to production with **XO only** but fully reshaped so Connect4 (Phase 4) drops in by adding rows, not redesigning UI. Three sprints, then prod cut. Single-game world during this work means picker filters / “add skill” modals collapse to one option visually — but the structure must be there so Phase 4 is additive.

Sprint	Items	Why grouped
3.8.A — Backend reshape + Profile (locked-step)	3.8.2.1, 3.8.3.1, 3.8.3.2, 3.8.3.3, 3.8.3.5, 3.8.6.4	POST /bots going skill-less breaks the existing Quick Bot wizard. The new Profile create-bot form + bot-card + “Add a skill” modal must ship in the same deploy. Bot-name uniqueness UX rides along (touches the same form). Manual Rusty-PvB regression check closes the sprint.
3.8.B — Gym + Profile→Gym navigation	3.8.3.4, 3.8.4.1, 3.8.4.2, 3.8.4.3	All Gym-side reshape. Sidebar bot→skill drilldown is the structural piece (visually trivial with one skill but required for Phase 4). “Train in Gym →” closes the navigation gap from 3.7a. Auto-repoint of <code>botModelId</code> on training completion makes “last-trained” accurate without manual select.
3.8.C — Play + Tournaments + closeout	3.8.2.5, 3.8.5.1, 3.8.5.2, 3.8.5.3, 3.8.5.4, 3.8.6.2, 3.8.6.3	Pickers move to identity-scoped (<code>botId</code> only); backend resolves (<code>botId</code> , <code>gameId</code>) at match start. Tournament reg rejects skill-missing bots. Rankings reframe as bot identity. Vitest + Playwright closeout. After this sprint the system is structurally Phase-4-ready.
Production cut	n/a	Once 3.8.C green on staging + smoke clean, deploy to <code>xo-*prod</code> . Connect4 (Phase 4) lands later as additive: one <code>BotSkill</code> row per built-in bot, one game package, no platform reshape.

Items keep their canonical 3.8.x.y numbering below — the sprint table above is just the execution overlay.

E2E test mandate (locked 2026-05-02): every sprint above ships its Playwright updates *in the same PR* as the user-surface change. Especially the journey/onboarding suite (`e2e/tests/journey*.spec.ts`) — the bot-create form, picker UI, and Gym sidebar are all directly asserted there. No “fix e2e next sprint” — broken e2e leaves the deploy gate dishonest. Run `cd e2e && npm test` against running services before signaling sprint complete.

3.8.1 Schema + seed

- ☒ Confirm `BotSkill (botId, gameId) unique + BotSkill.botId` relation to `User` hold up for multi-skill. No new migration expected. (*Verified 2026-04-26 — partial unique index (bot_id, game_id) WHERE bot_id IS NOT NULL exists at DB and Prisma level. BotSkill.botId is intentionally a loose String pointer matching the existing User.botModelId pattern; cascade safety handled in DELETE-bot route.*)

- ⊗ Update the Prisma seed to give each built-in bot (Rusty/Copper/Sterling/Magnus) exactly one `XO BotSkill` — behaviour-preserving, but structured so a Connect4 skill can be appended in Phase 4 by adding one row per bot. (*BotSkill.id matches botModelId so the existing string pointer + new row are self-consistent.*)
- ⊗ Keep `User.botModelId` as a nullable pointer to the bot’s *primary* (last-trained-or-selected) skill. (*Documented via Prisma /// doc comment on the field; semantics formalised. Wiring to update on train/switch lands with 3.8.4 frontend work.*)

3.8.2 Backend — bots + skills API

- ⊗ **[Sprint 3.8.A]** `POST /api/v1/bots` — accepts `{ name, avatarUrl, competitive }` only. No algorithm/game at this step. Creates a skill-less bot. `botModelId` starts null. (*Quick Bot wizard preserved via separate `POST /bots/quick` legacy path.*)
- ⊗ `POST /api/v1/bots/:botId/skills` — body `{ gameId, algorithm, modelType }`. Creates a `BotSkill`, sets `botModelId = newSkill.id` if it’s the bot’s first. Idempotent on `(botId, gameId)` — second call returns existing skill.
- ⊗ `GET /api/v1/bots/:botId` — includes `skills: BotSkill[]` with per-skill ELO joined from `GameElo` (`userId=botId, gameId=skill.gameId`).
- ⊗ `DELETE /api/v1/bots/:botId/skills/:skillId` — removes one skill. If the deleted skill was `botModelId`, repoint to any remaining skill or null. (*Also fixed `DELETE /api/v1/bots/:id` to sweep all the bot’s `BotSkill` rows, not just the primary.*)
- ⊗ **[Sprint 3.8.C]** Tournament registration: validates that the picked bot has a `BotSkill` for the tournament’s `gameId`; returns 400 with code: `'NO_SKILL'` if not. Logic in `tournament/src/lib/registrationGuards.js` `assertBotHasSkillForGame`, called from `POST /api/tournaments/:id/register`.
- ⊗ `GET /api/v1/bots?gameId=X` — list helper for community bot pickers, filters to bots that have a skill for X.

3.8.3 Frontend — Profile / bot creation

- ⊗ **[Sprint 3.8.A]** Profile “Create a bot” form: reduce to name + avatar + competitive flag. Submit → bot card appears with “No skills yet — Add a skill” affordance.
- ⊗ **[Sprint 3.8.A]** Bot card shows skill pills (one per game with `{ gameLabel, algorithm, ELO, episodes }`) and an “+ Add skill” chip. Clicking a pill opens Gym focused on that `(botId, gameId)` — deep-link via `/gym?bot=:botId&gameId=:gameId`.
- ⊗ **[Sprint 3.8.A]** “Add a skill” modal: pick game (dropdown of games that don’t already have a skill for this bot) + algorithm + optional starter config. Submits to `POST /bots/:botId/skills`. (*Single-game world: the game dropdown shows only XO; structure stays so Connect4 row lights it up.*)
- ⊗ **[Sprint 3.8.B]** Gym deep-link from Profile (gym-navigation gap close): each bot row gets a visible “Train in Gym →” button (deep-links to `/gym?bot=:botId&gameId=:firstSkillGameId`, or `/gym?bot=:botId` for skill-less bots), and a “Gym □” link sits next to “+ Create new bot” inside the My Bots section. Closes the 3.7a navigation gap.
- ⊗ **[Sprint 3.8.A]** Bot-name uniqueness UX (relocated from 3.7a.2): bot-create form checks availability inline (debounced `GET /bots/check-name`) before submit; per-owner dedup matches the partial-unique indexes (cross-owner collisions allowed); Rankings page renders “Rusty · @joe” / “Rusty · built-in” via `disambiguateBotLabels` when names collide in the visible set. (*Bot picker / tournament bracket helpers already in place — apply on those surfaces in Sprint 3.8.C alongside the picker reshape.*)

3.8.4 Frontend — Gym

- ⊗ **[Sprint 3.8.B]** Left sidebar shows bots; selecting a bot expands it to surface its `BotSkill` rows as a second-level list. Selecting a skill opens the existing Gym tabs scoped to that `(botId, gameId)` and keeps `?bot=...&gameId=...` in sync so the deep-link is shareable. (*Visually trivial with one skill per bot today; structural for Phase 4.*)
- ⊗ **[Sprint 3.8.B]** “Add skill for this bot” affordance inside Gym (header button on the active-skill panel + empty-state CTA on a skill-less bot) — shares the `ProfileAddSkillModal` and auto-selects the new skill on success.
- ⊗ **[Sprint 3.8.B]** When a training session completes, the backend auto-repoints `User.botModelId` to the just-finished `BotSkill` (`repointBotPrimarySkill`, exported from both `mlService` and `skillService`; covered by `vitest` in each service’s `__tests__`). The frontend `handleTrainingComplete` mirrors the same update optimistically so the Gym sidebar / Profile cache don’t show stale state until the next bots refetch.

3.8.5 Frontend — Play + tournaments

- ⊗ **[Sprint 3.8.C]** Community bot picker lists bots filtered server-side by `gameId` (`GET /api/v1/bots?gameId=:gameId` joins `BotSkill` to drop bots that lack a skill for that game). `communityBotCache.js` consumes it; no “Bot X for Game

Y” duplicates.

- ☒ **[Sprint 3.8.C]** Starting a game with a community bot resolves the skill server-side from (botId, gameId) at match start — picker payload carries only botId. useGameSDK, PlayPage, TablesPage, communityBotCache all dropped the legacy botSkillId field; the realtime route + tableFlowService.createHvbTable no longer accept it; missing skill → 400 NO_SKILL.
- ☒ **[Sprint 3.8.C]** Tournament registration picks a bot; backend resolves (botId, tournamentGameId) and rejects registration if the bot has no skill for that game (see 3.8.2.5 above).
- ☒ **[Sprint 3.8.C]** Rankings page already per-game via GameElo; renders bot *identity* (not skill) as the row with disambiguateBotLabels for cross-owner collisions and link to /bots/:botId. (Shipped in Sprint A.3 — verified here.)

3.8.6 QA + tests

- ☒ Vitest: POST /bots/:id/skills creates + repoints botModelId; idempotent on (botId, gameId); deleting the primary skill repoints to next remaining or null. (21 new cases in bots.test.js; the POST /bots skill-less-bot reshape is deferred so its case lands with that work.)
- ☒ **[Sprint 3.8.C]** Vitest: tournament/src/lib/__tests__/registrationGuards.test.js covers the four observable behaviours (human pass-through, bot-with-skill pass, bot-without-skill 400 NO_SKILL, gameId-keyed lookup). Pairs with the route wiring at tournament/src/routes/tournaments.js POST /:id/register.
- ☒ **[Sprint 3.8.C]** Playwright e2e/tests/open-items.spec.js §11i: user creates a bot, adds an XO skill, then starts a PvP match against a community bot — proves the two-step flow end-to-end with the new identity-scoped picker.
- ☐ **[Sprint 3.8.A]** Manual: Rusty with an XO skill still plays a PvP match identically to v1.28. (Sprint-A closeout regression check after the POST /bots reshape lands.)

Phase 4 — Connect4

Goal: Second game ships. Proves SDK + botInterface are reusable. Confirms npm workflow.

Prerequisite: Phase 3.8 (Multi-Skill Bots) shipped. Connect4 adds one BotSkill per built-in bot for gameId: 'connect4' — the identity layer is already in place.

4.1 Connect4 game package

- ☐ Create packages/game-connect4/
- ☐ Implement game logic (gravity mechanic, win detection: horizontal/vertical/diagonal)
- ☐ Implement full meta export including bot/training support flags
- ☐ Implement full GameSDK contract
- ☐ Implement botInterface — minimax bot, training support
- ☐ Build GymComponent for Connect4 training
- ☐ Implement focused and chrome-present rendering modes
- ☐ Build game UI (6x7 board, column drop animation, win highlight)
- ☐ Publish @calliduity/game-connect4 to GitHub Packages
- ☐ Verify platform loads Connect4 via React.lazy
- ☐ Connect4 appears in Tables page (create table, join, play)
- ☐ Gym and Puzzles tabs work for Connect4
- ☐ Replay works end-to-end for Connect4
- ☐ Connect4 ELO tracked in GameElo (separate from XO ELO)

4.3 TensorFlow.js evaluation (conditional)

Only if pure-JS training is noticeably slow for Connect4’s 6x7 board. See doc/TensorflowJS_Migration_Plan.md.

- ☐ Benchmark training speed for Connect4 with pure-JS engine
- ☐ If acceptable: defer TF.js, note for Poker/Pong evaluation
- ☐ If too slow: spike on TF.js migration using existing plan, complete before Connect4 ships

4.2 Pong real-time spike results review

Spike completed in Phase 1.8. This is the decision point.

- ☐ Review Phase 1.8 findings
 - ☐ Confirm Pong architecture (WebSocket loop or WebRTC)
 - ☐ Identify any SDK or botInterface changes needed for real-time games — resolve before Phase 6
-

Phase 5 — Poker

Goal: Third game ships. Introduces variable player counts, hidden information, and multi-player bot dynamics.

5.0 Multi-seat sit-down shell (relocated from 3.6)

Shell gains layouts for 3–8p sit-down tables and the per-seat render slot API for hidden-info games. No user-visible change from a gameplay standpoint — this is infrastructure so Poker can land cleanly. Was parked in Phase 3.6 until Poker was imminent.

- ☐ Seat position maps for 3/4/6/8p (round / oval) in the shell.
- ☐ Per-seat render slot API — game returns content per seat, shell positions it spatially.
- ☐ Per-seat visibility control via `getPlayerState(playerId)` — shell asks the game what to render for each seat from each viewer's POV.
- ☐ Responsive behavior: 8p oval on desktop, compact/rotated on portrait mobile.
- ☐ XO / Connect4 leave per-seat slots empty — no game-package changes required.

5.1 SDK extension — hidden information

- ☐ `sdk.getPlayerState(playerId)` is already in the contract — implement platform-side enforcement
- ☐ Platform SDK provider filters game state per player before delivering to each client
- ☐ Verify `@calllidity/game-xo` and `@calllidity/game-connect4` unaffected (public information games)

5.2 Table model — variable player count UX

- ☐ Verify forming status handles 2-7 seat tables correctly
- ☐ Update table creation UI to support configuring player count within game's min/max range
- ☐ forming state visible on Tables page — shows seats filled vs. waiting

5.3 Poker game package

- ☐ Create `packages/game-poker/`
 - ☐ Implement Texas Hold'em rules (pre-flop, flop, turn, river, showdown)
 - ☐ Implement virtual chip stack and betting rounds
 - ☐ Implement meta export (`minPlayers: 2, maxPlayers: 7, supportsBots: true`)
 - ☐ Implement `sdk.getPlayerState` — returns only what each player can see (hole cards hidden)
 - ☐ Implement `botInterface.makeMove()` — bot poker strategy
 - ☐ Implement `getPreviewState()` — community cards and pot size only
 - ☐ Build game UI (table layout, hole cards, community cards, betting controls)
 - ☐ Publish `@calllidity/game-poker` to GitHub Packages
 - ☐ Verify platform loads Poker via `React.lazy`
 - ☐ Poker appears in Tables page with forming/waiting state
 - ☐ Replay works — public actions only, folded hands not revealed
 - ☐ Poker ELO tracked in `GameELO`
-

Phase 6 — Pong

Goal: First real-time game ships on the architecture validated by Phase 1.8 spike.

6.1 Real-time infrastructure

- ☐ Implement game loop based on spike findings (tight WebSocket or WebRTC)

- ☐ Server-authoritative state with client-side interpolation
- ☐ Sampled snapshot storage for replay (100ms intervals)
- ☐ Handle reconnection and latency gracefully

6.2 Pong game package

- ☐ **SDK prerequisite (deferred from 3.5):** extend `GameMeta.tableArchetype` union to include 'head-to-head', and add `meta.orientations: ('horizontal' | 'vertical') []` for paddle axis. PlatformShell needs to render a head-to-head layout (opponents on opposite sides of the playfield, not across a table) when Pong's meta declares it.
- ☐ Create packages/game-pong/
- ☐ Implement game physics (ball, paddles, scoring)
- ☐ Implement meta export (`minPlayers: 2, maxPlayers: 2, supportsBots: true, tableArchetype: 'head-to-head', orientations: ['horizontal', 'vertical']`)
- ☐ Implement SDK contract adapted for real-time (continuous state updates)
- ☐ Implement `botInterface.makeMove()` — real-time paddle AI
- ☐ Implement `getPreviewState()` — current score and ball position
- ☐ Build game UI (canvas-based renderer, paddle controls)
- ☐ Publish @calllidity/game-pong to GitHub Packages
- ☐ Verify platform loads Pong via `React.lazy`
- ☐ Pong appears in Tables page
- ☐ Spectator live view works
- ☐ Replay works via sampled snapshot sequence
- ☐ Pong ELO tracked in GameElo

Phase 7 — External Developer Onboarding

Goal: Open the platform to approved third-party game and bot developers.

7.1 SDK + documentation

- ☐ Publish @calllidity/game-sdk as a standalone npm package (TypeScript types, helpers, full contract)
- ☐ Publish SDK + botInterface documentation publicly
- ☐ Write game developer guide: contract spec, SDK usage, botInterface implementation, publish workflow
- ☐ Write bot developer guide: makeMove contract, deployment options, ELO and tournament participation
- ☐ Define submission and review process for external games and bots
- ☐ Evaluate opening GitHub Packages scope or migrating to public npm registry
- ☐ Onboard first external developer (game or bot) as a pilot

7.2 XO as the registry-switch reference (relocated from 1.4)

Validates the URL-based dynamic-import path that external developers will use. The bundled path already works; this proves the same package can be consumed remotely.

- ☐ Publish @calllidity/game-xo to GitHub Packages — requires CALLIDITY_NPM_TOKEN secret (fine-grained PAT with write:packages on calllidity.org); see doc/Registry_Switch_Guide.md
- ☐ Deploy @calllidity/game-xo as a standalone local test service — requires importmap for shared React instance (documented in doc/Registry_Switch_Guide.md)
- ☐ Verify platform loads XO via dynamic URL import
- ☐ Confirm `/* @vite-ignore */ import` works, CORS headers correct, SDK props cross the bundle boundary correctly

Cross-Cutting Items (any phase)

- ☐ Admin panel: replay TTL settings (casual + tournament), table management, bot management
- ☐ Rankings page: per-game ELO display, filter by game, human vs bot toggle
- ☐ Tournaments: verify `isTournament: true` set on generated tables, replay link from bracket view

- Mobile QA: Tables page, game shell focused/chrome modes, Gym, replay controls
- Performance: verify `React.lazy` chunking — each game is a separate bundle
- Bot leaderboard: per-game bot rankings, ELO history, training stats
- **Rendered-table polish (relocated from 3.7).** Lands opportunistically once live use surfaces friction:
 - Sit-down animation on seat claim (Table_Paradigm §4.4 option C)
 - Showdown / chip / card animations for card games (per-game assets; arrives with Poker)
 - **ELO in seat pod** — shell fetches `GameElo` for each seated user on table load and displays the rating below the display name. Shell owns the fetch (not the game package) via a join on `GET /api/v1/tables/:id` or a dedicated enrichment endpoint. Deferred from 3.5 — display name + avatar is sufficient for the core paradigm.
 - Any additional visual refinements from live use
- **Cross-tab session-switch banner.** When a user signs in as a different account in another tab (same browser/origin), the cookies + localStorage session cache are shared, so every other tab silently swaps to the new user on the next `useOptimisticSession` poll (≤ 60 s). This is correct HTTP/cookie behavior — not fixable — but it is confusing. Detect when the `polledSession.user.id` differs from the previously-cached id on the same tab, and surface a banner (“Signed-in account changed in another tab — now signed in as **B**. Refresh to continue.”) instead of silently mutating the UI. Small polish; likely fits in 30 lines in `useOptimisticSession` + a banner component.
- **socket.io multi-machine routing on Fly.io.** As of 2026-04-16 staging has been manually scaled to 1 backend machine because `socket.io` polling transport + Fly.io round-robin load balancing causes a 400-Bad-Request cascade on every other poll (each polling request can land on a different machine, but only the issuing machine knows the SID). Current workaround: `flyctl scale count 1 -a xo-backend-staging`. Real fixes (pick one before scaling back to N machines in prod):
 - **Sticky sessions via fly-replay.** Add backend middleware that detects a polling request carrying an SID owned by a different instance and sets `fly-replay: instance=<owner>` so Fly’s edge proxy re-routes. Requires tracking SID-to-instance-id mapping (e.g., via Redis with the existing adapter).
 - **Restore WebSocket transport.** `landing/src/lib/socket.js` forces transports: `['polling']` because the landing express + http-proxy-middleware chain was dropping WS upgrades on Fly. Fix the landing proxy to forward upgrade + connection headers, re-enable WS on the client, and the multi-machine problem disappears (a WS stays on the machine it opened on). Lowest-risk long-term fix.
 - **Route /socket.io straight to backend from the edge** (skip landing proxy entirely). Requires a DNS/routing change; simplest if we move away from the double-proxy topology anyway.